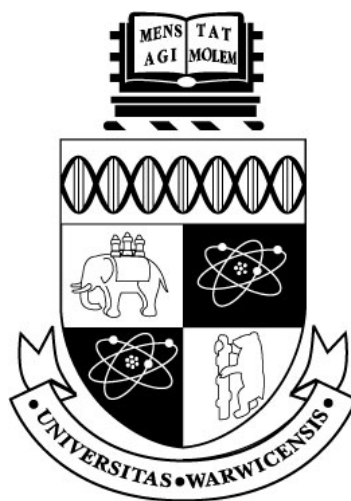




GODOT 4.6 大型可视化行为树的可读性 与运行时检查优化

by

[作者姓名]

A thesis submitted in partial fulfilment
of the requirements for the degree of

游戏工程理学硕士

UNIVERSITY OF WARWICK

华威大学 WMG

2026 年 9 月

目录

目录	ii
插图	v
表格	vi
致谢	vii
声明	viii
摘要	ix
1 引言	1
1.1 研究背景	1
1.2 问题定义与研究空白	1
1.3 研究目标与问题	2
1.4 研究目标与成功标准	2
1.5 主要贡献	3
1.6 论文结构	3
2 文献综述	4
2.1 作为可执行层级的行为树	4
2.2 节点-连线树与扩展性	4
2.3 焦点、上下文与概览	5
2.4 复杂度管理与心理地图	5
2.5 运行时解释与失败定位	6
2.6 研究空白与设计启示	6
3 研究方法	7
3.1 研究策略	7
3.2 实验因素与数据集	7
3.3 受控多规模显示与拖拽实验	8
3.4 物理屏幕尺寸实验	8

3.5	可玩树生态显示校验	9
3.6	正确性与集成评估	9
3.7	视觉验证	9
3.8	可选未来人体验证	10
3.9	有效性与可复现性	10
4	系统设计与实现	11
4.1	架构与项目边界	11
4.2	资源模型与校验	11
4.3	运行时语义	12
4.4	编辑器编写工作流	12
4.5	Live Debug 与黑板检查	13
4.6	大型树显示机制	13
4.6.1	信息密度控制	13
4.6.2	焦点与结构裁剪	14
4.6.3	导航与概览	14
4.6.4	连线与运行表示	15
4.7	物理屏幕实验基础设施	15
4.8	运行时拓扑缓存	15
4.9	演示游戏	15
5	结果与分析	17
5.1	功能正确性	17
5.2	受控多规模显示结果	18
5.3	物理屏幕尺寸结果	18
5.4	版本化拖拽交互结果	19
5.5	可玩 241 节点树显示结果	20
5.6	路径与概览结果	21
5.7	视觉回归发现	21
5.8	运行时缓存结果	22
5.9	人体可用性证据边界	23
5.10	对应研究问题的结果	23
6	讨论	24
6.1	完整系统作为研究工具	24
6.2	显示结果的解释	24
6.3	Live Debug 作为可读性机制	25
6.4	缓存权衡	26
6.5	可靠性与发布	26

6.6 有效性威胁	26
6.6.1 构念有效性	26
6.6.2 内部有效性	26
6.6.3 外部有效性	27
6.6.4 研究者与人体有效性边界	27
6.7 实践启示	27
7 结论	28
7.1 总结	28
7.2 贡献	28
7.3 局限	29
7.4 未来工作	29
7.5 最终结论	29
参考文献	30
A 功能参考	32
B 复现与证据索引	33
C 可选未来参与者实验	34
D 初稿完成清单	35

插图

4.1	Godot 编辑器、行为树资源、Runner 与 Actor 之间的插件架构和数据流。	11
4.2	仅位于编辑器中的活动路径和失败原因显示。	14
4.3	同一行为树的完整细节基线和紧凑显示。	14
5.1	五种受控规模下的密度、显著性和上下文降低。	19
5.2	焦点 + 上下文、结构裁剪和显著性导航的实现示例。	21
5.3	开启与关闭安全拓扑缓存时的 Tick 扩展。	22

表格

- 4.1 已实现的运行时节点语义。 12
- 5.1 最终核心自动回归。 17
- 5.2 五种资源规模的受控显示测量，降低比例相对 Baseline。 18
- 5.3 三种物理屏幕尺寸下的复杂树覆盖率。分辨率仅为审计元数据，不是实验处理。 19
- 5.4 优化前与当前版本的固定路径拖拽处理时间，中位数 [Q1, Q3]。 20
- 5.5 可玩 241 节点行为树的客观显示结果，降低比例均相对 Baseline。 20
- 5.6 活动分支淡化与增强小地图。 21
- 5.7 按树规模和 Runner 数统计的运行时中位值。 22
- A.1 最终编辑器的独立显示功能。 32

致谢

此处预留给作者感谢导师，以及帮助测试软件和审阅论文的人。提交前应由作者自行完成最终文字。

声明

本论文作为本人申请游戏工程理学硕士学位的材料提交给华威大学。论文由本人完成，且未曾用于申请其他学位。除明确引用的资料外，本文所述软件、测试数据和分析均来自本项目。本文没有报告任何参与者实验结果；可选的未来人体对比协议仅作为支持材料保留。正式提交前，作者必须根据学校规定审阅并个性化本声明，包括如实说明生成式人工智能的使用方式。

摘要

可视化行为树常用于编写游戏智能体的决策，但随着节点数量以及运行时信息增加，传统节点-连线编辑器会变得难以检查。本文设计并评估了一个 Godot 4.6 编辑器插件，将完整、可执行的行为树系统与可独立开关的大树显示和实时调试技术结合。运行时支持有序、响应式、随机、并行、重复和定时控制流，支持 Actor 方法调用、类型化黑板、Decorator 以及仅存在于编辑器中的运行检查。编辑器实现紧凑卡片、语义缩放、鱼眼焦点 + 上下文、子树折叠与聚焦、搜索高亮、概览 + 细节、稳定布局、多种连线、当前路径强调和失败原因标注。

评估使用同一生成器构造的 31、61、121、241 和 364 节点树、一个驱动可玩敌人的 241 节点树、自动语义与 GUI 测试以及真实 GPU 视觉回归。五种规模下的六种显示条件各经过三次预热并重复 30 次，共获得 900 条观测。Compact 在不隐藏卡片的情况下将卡片面积降低 61.09–61.35%，语义概览降低 90.27–90.34%，Search 淡化 96.15–99.67% 的卡片，所有条件均保持零重叠、保存位置和执行顺序。版本化 A1–B1–B2–A2 对照另记录 200 次固定 240 步拖拽；五种规模中四种的中位处理时间下降，241 与 364 节点的 Bootstrap 区间完全高于零，364 节点中位时间从 2,939.47 ms 降至 1,758.30 ms (40.18%)。但 31 节点点估计回退且区间跨零，因此预设严格判据未通过。另一组 270 条观测以每厘米 35 个逻辑单位的相同密度比较三台已连接物理屏幕。从 15.94 英寸增至 31.55 英寸时，241 节点 Baseline 适配后覆盖率由 18.32% 增至 46.53%，364 节点由 12.13% 增至 30.82%；但所有复杂树仍达到 0.10x 最小缩放。可玩资源复现了几何效果。这些是客观表示与交互结果，不证明人的理解能力；人体可用性仍未验证。

1 引言

1.1 研究背景

现代游戏智能体需要在巡逻、追击、攻击、撤退和恢复等行为之间做出选择。对于小型原型，把全部决策写在单一过程式脚本中是可行的；但随着状态和中断条件增加，控制流会越来越难以检查和修改。行为树（Behaviour Tree, BT）把决策逻辑表示为控制节点和叶节点组成的层级结构。复合节点决定如何选择子节点，条件和动作则把抽象决策结构连接到游戏状态与 Actor 代码。高层策略可以独立于移动、动画或战斗方法进行编辑，这是行为树在游戏开发中的主要价值之一 (Colledanchise and Ögren, 2018)。

商业引擎已经证明可视化行为编写的实用性。Unreal Engine 将节点-连线树、Decorator、黑板和编辑器运行状态结合 (Epic Games, 2026)。Godot 4.6 提供 EditorPlugin、GraphEdit 和 GraphNode 等通用扩展接口，但并没有规定完整的可视化行为树工作流 (Godot Engine contributors, 2026b,a)。因此本项目同时面对工程问题 and 研究问题：插件首先必须能正确执行行为树并接入任意 Actor；在此基础上，随着节点和运行标注增加，图形表示还必须保持可检查性。

传统节点-连线树能够直接表达父子拓扑，但会随宽度和深度增长而迅速占用空间。行为树卡片还可能显示参数、描述、Decorator、黑板键和执行状态，比只显示标签的普通层级图更大。当数十或数百张卡片放入有限画布时，开发者必须频繁缩放、平移和搜索，当前执行路径也可能淹没在非活动分支中。这不只是美观问题，因为节点图正是设计者定位任务、理解顺序、修改条件和诊断失败的主要工具。

因此，本项目把一个功能完整的 Godot 行为树插件作为大型树显示研究的平台。在保持上下端口和“同级从左到右执行”的语义基础上，加入可独立开关的焦点 + 上下文、复杂度管理、导航和运行时解释技术。每项技术都能单独关闭，一方面防止视觉功能故障拖垮编辑器，另一方面保留可复现的基线比较条件。

1.2 问题定义与研究空白

树和图可视化领域已经提出许多方法。鱼眼和焦点 + 上下文视图会为焦点附近的元素分配更多空间 (Furnas, 1986; Lamping et al., 1995)；概览 + 细节保留全

局参考视图 (Cockburn et al., 2008); 多列布局可以改善大扇出树的浏览 (Song et al., 2010); 增量展开和折叠可以降低图复杂度, 同时尽量保持用户的心理地图 (Dogrusoz et al., 2018; Misue et al., 1995); 大型分层图研究还表明, 路径任务的效率取决于具体表示和任务 (Bae and Watson, 2011)。

然而, 这些研究没有直接回答游戏行为树编辑器应如何组合上述机制。行为树与文件目录等普通层级不同: 同级顺序具有执行语义; 叶节点会调用代码; Decorator 可能阻止分支运行; 图的状态会在每个运行 Tick 中变化。一个可用的方案必须保持执行顺序, 支持编辑与撤销, 公开黑板状态, 并在不向游戏画面加入调试覆盖层的前提下解释失败。现有研究可以指导单项设计, 但仍需要一个集成、可执行且可复现评估的 Godot 实现。

1.3 研究目标与问题

本文目标是设计、实现并评估一个 Godot 4.6 可视化行为树插件, 使其既能驱动真实 NPC, 又能为大型树的视觉和交互复杂度提供可测量的控制机制。

研究问题如下:

RQ1: Godot 编辑器插件能否提供控制非简单二维游戏智能体所需的核心编写、运行时、黑板、Decorator 和实时调试能力?

RQ2: 在不同受控树规模与物理屏幕尺寸下, 可独立开关的显示技术会如何改变可测表示属性和固定路径编写交互, 同时保持保存结构与执行顺序?

RQ3: 在不改变执行结果、并能安全处理资源原地修改的前提下, 拓扑缓存能否降低不同树规模和 Actor 数量下的 Tick 成本?

RQ2 有意限定为仪器化的表示和交互证据。几何、显著性、重叠、物理表面覆盖率与固定输入处理时间可以说明编辑器随树规模或可用显示面积变化时具体改变了什么, 却不能证明真人理解更快、错误更少或认知负荷更低。平衡的参与者协议仅作为可选未来构念验证保留, 不作为已完成证据, 也不虚构参与者结果。

1.4 研究目标与成功标准

项目的具体目标包括: 实现可序列化的树、节点、Decorator 和类型化黑板资源; 实现 Root、Sequence、Selector、Reactive Selector、Random Selector、Parallel、Repeat、Wait、Action 和 Condition 的 SUCCESS、FAILURE 与 RUNNING 语义; 允许可复用组件为 NPC 指定树和 Actor, 并由叶节点调用 Actor 方法; 提供创建、连接、断开、拖拽、类型化编辑、校验、保存、加载、撤销和重做; 只在 Godot 编辑器中显示路径、节点状态、黑板和失败原因; 为大型树显示方法提供独立

持久化开关和安全复位路径；构造确定性大树与复杂二维场景；最后以原始数据评价正确性、几何显示、物理屏幕尺寸稳健性、视觉回归、跨规模拖拽交互和洁净安装，并保留平衡开发者任务协议用于可选的人体验证。

成功标准不仅是插件能够启动。每类运行节点需要聚焦语义测试，编辑操作必须通过保存/重载和历史测试，游戏场景必须展示索敌、追击、攻击、搜索、撤退与恢复，视觉功能必须在真实渲染器上检查，缓存与非缓存执行必须得到相同终态。即使断言计数通过，只要日志出现非零退出、脚本错误、泄漏、原生崩溃或非法内存访问，仍判定测试失败。

1.5 主要贡献

本文的主要贡献为：

- 一个可分发的 Godot 4.6 行为树插件，包含可执行复合节点、叶节点、Decorator、Actor 方法集成和类型化黑板；
- 一个仅位于编辑器中的实时调试桥，可报告活动路径、黑板值和失败原因，而不把调试 UI 耦合到游戏；
- 一组集成的 19 项可独立开关显示功能，覆盖紧凑/语义表示、焦点 + 上下文、子树裁剪、导航、路径强调、无障碍编码和连线显示；
- 一套可复现评估材料，包括 31、61、121、241、364 节点资源、可玩的 241 节点树、真实 GPU 截图、900 条重复显示观测、200 条版本化拖拽观测，以及三台已连接屏幕上的 270 条物理屏幕观测，并提供原始 CSV 和确定性分析脚本；
- 一种精确拓扑签名缓存，可检测相同节点数量下的换序、改父级、Decorator 改归属和资源实例替换，并包含九组规模/负载测量。

1.6 论文结构

第2章批判性回顾行为树和可扩展层级可视化；第3章定义设计科学与实验方法；第4章说明资源、运行时、编辑器、显示机制、游戏场景和测试架构；第5章报告功能、几何、视觉与运行时结果；第6章结合研究问题和文献讨论结果与局限；第7章总结贡献和未来工作。附录提供功能、证据与复现实验索引。

2.1 作为可执行层级的行为树

行为树会重复评估一棵有根层级，并传播 `SUCCESS`、`FAILURE` 或 `RUNNING` 三种状态。`Sequence` 通常在第一个失败或运行中的子节点停止；`Selector` 在第一个成功或运行中的子节点停止；叶节点读取状态或执行动作。这个小型状态接口使复杂逻辑能够组合，并能明确表达中断策略 (Colledanchise and Ögren, 2018)。

三状态接口看似简单，却包含关键实现选择。运行中的 `Sequence` 可以记住当前索引，而 `Reactive Selector` 需要重新检查高优先级条件并抢占较低优先级分支；`Random Selector` 在子节点运行期间应保持同一次选择；`Parallel` 需要成功和失败阈值；`Repeat` 与 `Wait` 需要在重启或中断时清除记忆。因此，编辑器显示和运行语义不能完全分离：子节点顺序、Decorator 归属和节点身份都会影响执行，而且必须正确序列化。

黑板在感知、条件和动作之间提供共享键值状态。Unreal Engine 的工作流说明了把黑板、Decorator 和运行观察结合的价值 (Epic Games, 2026)。但本项目没有必要重现商业系统的全部能力。合理基线是：系统能够表达有意义的决策、调用项目 Actor 方法、验证黑板键，并解释分支为何运行或被拒绝。

2.2 节点-连线树与扩展性

节点-连线图适合行为树，因为能直接表示祖先关系和同级分组。经典整洁树算法追求层级一致、子树不重叠且相对紧凑的布局 (Reingold and Tilford, 1981)。`SpaceTree` 又把节点-连线结构与动态布局、缩放和选择性展开结合，其评估说明表示和交互会共同影响拓扑理解与重访任务 (Plaisant et al., 2002)。

问题在于规模。卡片占据面积，随着宽度和深度增加，显式连线也会变密。Song 等比较传统、列表和多列视图，发现多列界面在其大扇出层级的浏览和理解任务中更快，总体偏好也更高 (Song et al., 2010)。这一结果适用于包含大量备选行为的 `Selector` 或 `Sequence`，但不能直接照搬：论文中的子节点按字母排序，而行为树的横向顺序可能决定优先级。因此多列行为树必须明确显示顺序，且不能在布局时静默修改资源。

Bae 与 Watson 为大型分层图提出 `Quilts` 表示 (Bae and Watson, 2011)。其价值不一定是要求行为树完全改用矩阵，而是证明路径任务可能更适合另一种补

充表示。本项目因此保留可编辑节点图，同时加入紧凑文字路径和强活动路径编码，以针对运行时路径定位任务。

2.3 焦点、上下文与概览

Furnas 用兴趣度函数形式化了广义鱼眼视图：综合元素先验重要性和与焦点的距离，决定显示显著程度 (Furnas, 1986)。Lamping 等使用双曲几何在有限区域显示大型层级，对焦点进行放大并压缩远处上下文 (Lamping et al., 1995)。行为树编辑器可以借此读取鼠标所在节点，同时保留周边结构。

但是焦点 + 上下文并不天然优于其他方法。畸变会移动目标并增加点击难度，缩放变化也可能破坏心理地图。Cockburn 等区分概览 + 细节、缩放和焦点 + 上下文，并指出效果取决于任务、实现和切换成本 (Cockburn et al., 2008)。因此本项目把鱼眼最大倍率限制为 1.20，改变卡片内部表示而不是施加不稳定的整体图变换，以节点中心补偿位置，并在关闭时恢复全部几何。同时提供独立固定小地图，使两类技术可以比较或组合。

语义缩放与几何缩放不同。几何缩放改变物理大小，语义缩放改变显示属性。低细节卡片保留类型颜色、图标和短标题；高细节卡片展示方法、参数、描述和 Decorator。它以信息完整性换取密度，却不改变执行图几何。此前插件曾把滚轮与临时节点缩放耦合，造成滚动时节点先缩小再放大；分离信息可见性和图几何后，该问题被消除。

2.4 复杂度管理与心理地图

当不需要同时查看全部节点时，展开-折叠和隐藏-显示可以降低复杂度。Dogrusoz 等提出大型网络的增量复杂度管理方法，包括展开、折叠和布局更新 (Dogrusoz et al., 2018)。该研究与心理地图原则一致：如果一次编辑引起大量无关节点移动，用户就必须重新学习位置 (Misue et al., 1995)。行为树折叠应保留折叠根，报告隐藏内容，并尽量不移动无关分支。

子树折叠和子树聚焦解决不同问题。折叠用摘要替代后代，同时保留树的其他部分；聚焦只显示目标分支和必要上下文。二者都可能隐藏运行节点，因此插件提供隐藏节点数量、两层摘要、路径指示以及显式展开/聚焦控制，而不是无提示地删除视图信息。

搜索高亮与非活动分支淡化属于较柔和的复杂度管理：几何保持不变，只降低非相关内容显著性。这符合 Shneiderman 的“先概览、再缩放和过滤、按需细节”原则 (Shneiderman, 1996)。形状/图标和色盲友好配色则提供冗余通道，避免只用颜色表达节点类型。

2.5 运行时解释与失败定位

实时调试为静态层级增加时间维度。只高亮叶节点不能解释其到达路径；高亮所有历史节点又会混淆当前状态。本项目记录有序的 Root 到叶节点链条、逐节点状态和当前失败解释。非活动分支可以淡化，独立路径摘要在图缩小时仍可阅读。

Decorator 让失败解释更加重要。Action 本身可能正确，却因为黑板比较、冷却或时间限制而从未执行。在所属节点上显示 Decorator 摘要并附加失败原因，可以把运行证据连接回编写表示；实时黑板进一步显示 Key、运行时类型、值和 Schema 状态。这体现了分层图研究的任务导向原则：调试时用户往往在问“为什么这个分支没有运行”，而不是“完整拓扑是什么”。

2.6 研究空白与设计启示

文献支持多个单项机制，却没有规定如何把它们集成到一个可执行 Godot 行为树编辑器中。本文据此采用四项原则：保留节点-连线拓扑和明确同级顺序作为基线；分离几何放大、语义细节和结构过滤；在折叠与布局中维护空间上下文，并在隐藏细节时提供概览或路径表示；最后，把几何效果和软件正确性与人的任务表现分开评估。最后一项尤其重要：面积或可见节点减少只能证明表示更紧凑，不能证明更容易理解。

3 研究方法

3.1 研究策略

本研究采用设计科学与软件工程结合的策略。研究产物是 Godot 4.6 插件和可执行演示；研究知识来自把行为树语义和可视化文献中的设计要求落实为系统，再跨受控规模测量其正确性、显示属性和编写交互。运行时性能仅保留为次要工程证据，而不是主要成功标准。迭代开发是必要的，因为编辑器集成暴露了单独算法中看不到的问题，例如坐标空间错误、残留 Transform、重复连线和半写入调试快照。

过程分为五个阶段：首先把导师会议要求和论文规范转化为验收门槛；其次建立最小可执行行为树核心和游戏集成；然后加入编辑操作和 Live Debug；随后将显示技术放在独立开关后，每次增加功能都测试关闭与复位；最后用确定性工作负载和分层测试比较完成系统与自身基线。后续任务完成后重新运行完整回归，以防图重建或资源复制改动破坏早期功能。

3.2 实验因素与数据集

五棵确定性生成树分别包含 31、61、121、241 和 364 个资源节点，覆盖小图、导师所提约 50 节点问题以上的区域，以及逐步增大的压力场景。所有规模使用相同的广度优先结构规则、节点类型循环和附着 Decorator 比例，分别形成 26、51、101、202 和 305 张画布卡片，使规模成为主要受控差异。另有一棵生态有效性测试树包含 241 个资源节点，其中 202 个为画布卡片，39 个为附着的 Decorator。与生成树不同，它就是可玩敌人实际加载的资源，包含恢复、方向近战、越障跳跃、梯子攀爬、远程投射物、压迫推进、追击、最后位置搜索、返回、巡逻和待机行为。

主要受控实验比较 Baseline、Compact Cards、Optimized Overview、Optimized Search、Subtree Focus 和 Context Collapse；Overview 组合 Compact 与低细节 Semantic Zoom，Search 再加入唯一固定目标。Fisheye、Enhanced Minimap 和其他开关由独立功能与视觉回归测试评价。指标包括：渲染卡片相对基线卡片的可见比例；所有卡片宽高乘积之和；Fit-to-view 缩放和适配后卡片中心位于画布内的数量；卡片重叠对数和最小父子垂直间距；语义层保留字段；Search 淡化数；操作时间。

这些是界面仪器指标，不是心理指标。它们测量信息密度、上下文和固定输入处理，不测量人的理解、错误或认知负荷。

3.3 受控多规模显示与拖拽实验

受控实验在 NVIDIA GeForce RTX 5070 Laptop GPU 上使用 Godot 4.6 OpenGL Compatibility 渲染器和 1600×900 的 SubViewport。每个规模-条件组合先预热三次；30 个完整正式轮次同时循环规模与条件顺序，共产生 (5×6×30=900) 条观测。每次都从完全展开树、空查询、100% 缩放和无焦点状态开始，再应用一种条件。固定唯一的 Search 与 Focus 目标避免测量期间的人为选择。

脚本记录资源/卡片数、可见比例、Fit 缩放、适配后视口卡片、图边界、卡片面积、重叠、父子间距、字段、淡化、目标取景和交互时间。时间汇总报告中位数、Q1、Q3 和 P95，但不按时间给不同语义操作排名。预设断言要求规模精确、基线渲染全部卡片、所有条件零重叠且父子间距非负、保存位置和执行顺序不变；Compact 面积至少减少 40%，Overview 至少减少 70%，61 节点以上 Search 至少淡化 95% 的非目标卡片，121 节点以上 Focus 与 Collapse 至少减少 70% 的卡片。

同任务拖拽实验比较优化前提交 5d914ae 和当前系统。A1 为旧版升序，B1 为当前版降序，B2 为当前版升序，A2 为旧版降序。每个规模每区块先进行一次不计入统计的热身，再记录十次 240 采样点拖拽，因此每个版本-规模组合有 20 条观测，总计 200 条。脚本验证移动距离、最终资源同步和执行顺序；报告中位数 [Q1, Q3]、P95，以及固定种子 20260822 的 10,000 次 Bootstrap 中位降低区间。预设总体判据为至少 4/5 个规模点估计降低，且任何规模不得回退超过 5%。该实验测量固定路径编辑器处理，不是行为树 Tick 或人的流畅感知。

3.4 物理屏幕尺寸实验

另一个配对实验考察物理显示面积，并且不把原生像素数作为自变量。Windows EDID 数据识别出三台已连接屏幕，物理尺寸分别为 34×22 cm (15.94 英寸)、60×33 cm (26.96 英寸) 和 70×39 cm (31.55 英寸)。实验按每厘米 35 个逻辑单位的统一密度将三块物理表面转换为 1190×770、2100×1155 和 2450×1365 的逻辑画布。原生分辨率、刷新率、操作系统缩放和 GPU 连接只保留在设备清单中作为审计元数据。

相同的五棵确定性树和六种条件在每台物理屏幕上执行。每个“屏幕-规模-条件”单元先进行两次不记录的预热，再记录三次正式重复，因此获得 $3 \times 5 \times 6 \times 3 = 270$ 条观测。实验窗口移动到已映射的 Godot 屏幕索引，而真实 GPU SubViewport 使用按厘米归一化的画布。另保存 18 张截图，覆盖三台屏幕上 241 与 364 节点的 Baseline、Optimized Overview 和 Optimized Search。

主要指标为 Fit 前后画布内卡片中心数、适配后覆盖率、Fit 缩放、图面积与物理屏幕面积之比、换算为 cm^2 的卡片面积、搜索目标取景、上下文降低、重叠和父子间距。各条件还必须保持资源位置和从左到右执行顺序。操作时间为复现而保留，但因物理设备、显示链路和调度并非受控时间处理，不作跨屏幕性能比较。因此允许的推论只是配对几何屏幕尺寸效应，不是显示性能或人的可读性改善。

3.5 可玩树生态显示校验

独立的 241 节点可玩资源使用相同六种条件和硬件。每个条件预热三次并以循环顺序记录 30 次，共 180 条观测；Search 固定定位 Patrol Route Choice，Focus 固定选择 11 Layered Patrol。它用于检查受控效果能否在不规则且具有游戏意义的资源中复现，不作为第二个独立总体样本。

运行时实验交叉三种树规模、1/10/50 个 Runner 和缓存开/关。每个组合预热后测量七次。测试调整 Tick 数，使各条件执行一个有界且大致可比的总工作负载。使用 Tick 均值的中位数，以降低临时调度噪声影响；缓存和非缓存还必须返回相同终态。

3.6 正确性与集成评估

测试分为五个核心层。资源/运行时测试检查结构、节点语义、Decorator、Actor、Schema、调试快照和缓存失效；编辑器 GUI 测试实例化插件并检查创建、拖拽、连接、类型字段、保存重载、撤销重做和开关；基础游戏测试检查玩家与敌人交互；复杂竞技场测试覆盖索敌、响应式防御、方向近战、远程投射物、障碍感知与跳跃、对高处玩家的梯子追击、丢失目标、撤退与治疗；真实 GPU 回归在 1600×900 分辨率捕获 Godot Dummy 渲染器无法提供的可视属性。

判定策略严格：非零退出码、FAIL:、ERROR:、SCRIPT ERROR:、泄漏、资源未释放、非法访问或原生崩溃都会使套件失败。警告则按语义判断：故意调用不存在 Actor 方法是安全测试，应返回 FAILURE；-quit-after 后的文件扫描中断属于预期关闭行为。

3.7 视觉验证

自动断言不能发现所有显示故障。因此视觉协议在 NVIDIA GeForce RTX 5070 Laptop GPU 和 Godot OpenGL 3.3 Compatibility 渲染器上生成稳定截图。核心截图覆盖基线、类型编码、灰度、无障碍配色、单线、紧凑、鱼眼开关、折叠、搜索、运行失败、实时黑板、Schema 编辑、Key 选择、正交连线、捆绑连线和复杂树；另外四张固定截图覆盖 121 节点练习树、可玩 241 节点树概览、平衡搜索目标和确定性的七节点游戏路径。

人工检查关注卡片重叠、端口位置、重复边、文字裁切、残留缩放、视口居中、活动路径连续性、淡化复位和小地图覆盖。正是该方法发现鱼眼虽然状态变量正确，却存在偏右中心和滚轮缩放干扰。

3.8 可选未来人体验证

附录保留一套可选的被试内协议，用于测试自动指标无法回答的构念。它在可玩 241 节点树上比较 Baseline、Compact、Collapse、Fisheye、Search 和 Minimap，包含 Action 定位、活动路径报告和 Decorator 编辑；12 名参与者将产生 216 次正式试验。

目标、方法顺序和任务顺序已经平衡，结果可记录上限完成时间、成功率、误选、导航和七点评分。正式招募仍需学校伦理和导师批准。工作簿当前为零观测，因此该协议不为当前研究问题提供结果，本文不报告任何推断性人体结论。

3.9 有效性与可复现性

固定资源、确定性目标、独立开关、900 条循环顺序显示观测、270 条配对物理屏幕观测、A1-B1-B2-A2 拖拽顺序和集成后全回归增强了内部有效性。构念有效性仍受限制，因为面积、物理表面覆盖率和字段数测量的是表示属性，不是可读性本身。外部有效性限于一台 Windows 电脑、Godot 4.6、按可用性选择的三台屏幕、固定生成资源和一个二维游戏。因此所有时间值都与硬件、渲染器、视口、预热和工作负载一起报告，不视为跨设备常数。

复现材料包括 900 条显示数据、200 条拖拽数据、270 条物理屏幕数据、分组 CSV、SHA-256 清单、固定种子分析脚本、截图、测试脚本、洁净安装打包脚本和 MIT 许可证插件包。物理屏幕实验使用只追加 Session，以便以后接入新屏幕后新增证据而不覆盖当前三屏数据。可选人体实验材料与自动证据分开保存，防止未采集的数据被误认为结果。

系统设计4与实现

4.1 架构与项目边界

系统分为编辑器层、资源层、运行时层和验证层（图4.1）。活动演示项目与内容同步的分发插件模板相互分离。游戏场景只依赖序列化资源和运行组件；编辑器控件与 Live Debug 表示不进入游戏 GUI。这一边界避免导出后的智能体行为隐式依赖编写工具。

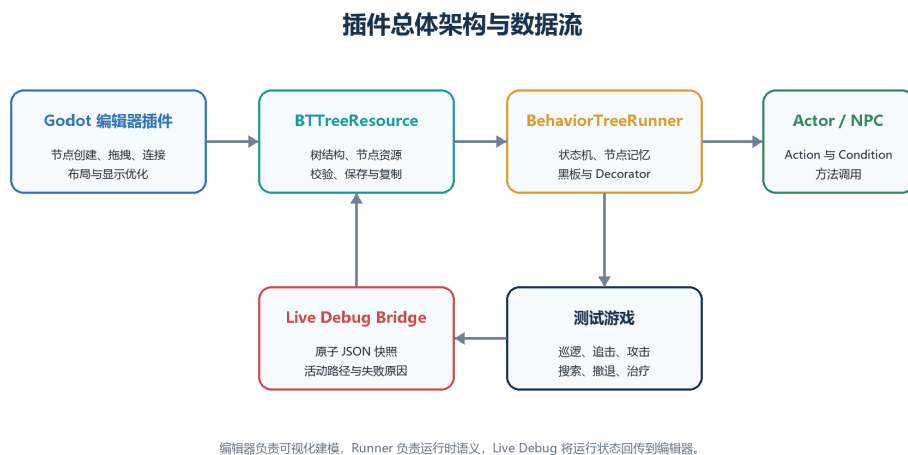


图 4.1: Godot 编辑器、行为树资源、Runner 与 Actor 之间的插件架构和数据流。

BTreeResource 保存 RootID、节点数组和可选黑板 Schema; BTreeNodeResource 保存类型、标题、描述、参数、图位置、普通父级和 Decorator 所有者; BehaviorTreeRunner 针对 Actor 与字典黑板执行树; BehaviorTreeComponent 把树、ActorPath、Tick 模式和生命周期暴露为可复用场景组件。编辑器中，BTEditorView 管理 Toolbar、Graph、Inspector、Schema Editor、历史和 Live Debug，另外两个控件负责画布输入、连线和卡片显示。

4.2 资源模型与校验

普通层级使用 parent_id; Decorator 使用 decorator_parent_id，因此不会成为普通控制流子节点。子节点按 X 坐标排序，X 相同时按 Y 排序，使画面从左到右与执行顺序一致。移动同级卡片可以有意改变优先级，序列化则保存布局。

校验拒绝缺失 Root、重复 ID、无效父级、非法 Decorator 归属和节点专属结构错误。Root 不得有父级，叶节点不得拥有普通子节点，Decorator 必须依附有效普通节点，必需参数也必须存在。保存和测试使用同一校验入口。

黑板 Schema 支持 Bool、Int、Float、String 和 Vector2 类型、默认值和动态键策略。严格模式报告空引用和未声明键。Inspector 同时提供带类型的键选择器和自由输入；反向索引可以列出每个键的引用节点和未使用声明。Schema 修改通过深拷贝节点与条目进入树级撤销重做。

4.3 运行时语义

每次 Tick 返回 SUCCESS、FAILURE 或 RUNNING。节点记忆按 ID 保存，并在重启、换树或分支中断时清理。表4.1概括节点语义。

表 4.1: 已实现的运行时节点语义。

节点	行为
Root	执行唯一普通子节点。
Sequence	按序执行，遇失败或运行停止，并记忆运行索引。
Selector	按序执行，遇成功或运行停止，并记忆运行索引。
Reactive Selector	每 Tick 从高优先级重新检查，可中断低优先级分支。
Random Selector	使用可设种子的随机选择，运行期间保持同一子节点。
Parallel	Tick 多个子节点并应用成功/失败阈值。
Repeat	固定次数或无限重复，跨 Tick 保存计数。
Wait	累积 delta，到达时间后成功。
Condition	调用 Actor 判断或比较黑板值。
Action	调用 Actor 方法，把 Bool 或状态转成树状态。

Action 和 Actor 模式 Condition 统一调用 method(blackboard, delta, node)。缺失方法会安全失败并生成调试原因，而不是触发致命脚本错误。黑板条件支持存在、不存在、真假、相等、不等和数值大小比较。

Decorator 在所属节点前或周围执行，支持黑板比较、Cooldown、Time Limit、Invert、Force Result 和 Repeat。Cooldown 防止立即重入，Time Limit 在超时后使运行子节点失败，Invert 交换成功与失败，强制结果只覆盖终态而保留运行态。子树复位同时清理 Decorator 和子节点记忆。

4.4 编辑器编写工作流

插件注册为底部面板。为避免永久显示一排节点创建按钮，用户在画布上右键，通过上下文菜单把所需节点创建在对应图位置。节点可以移动、从底部方

形端口连接到顶部方形端口、断开、复制和删除；复合节点支持多个子节点；右键菜单还提供常用操作和子树显示命令。行为树选择器只显示已选资源名称，不要求用户管理原始路径文本框。所有数据改动都推入树快照，因此撤销重做覆盖创建、删除、移动、连线、参数、Schema 和折叠状态。

Inspector 为常见字段提供类型化控件，不要求手写 JSON。Action 编辑方法名；Condition 编辑 Actor/Blackboard 模式、Key、Operator 与值；Random、Parallel、Repeat、Wait 和 Decorator 暴露对应策略。Advanced JSON 仍作为兼容入口，无效 JSON 不会持续输出红色错误，并在修复前保留旧有效参数。

连线需要特殊处理：Godot 原生 GraphEdit 曲线使用侧面 Slot，而本项目采用上下端口。单连接模式隐藏原生持久层并绘制自定义上下边，拖拽预览期间临时恢复原生层。自定义线段命中保留右键断开和撤销重做；关闭该功能则恢复原生线，形成安全回退。

大型树拖拽避免在每个指针事件上发送资源变更通知。手动拖拽期间卡片只更新视觉位置并请求连线重绘，在释放鼠标后才把最终位置同步到资源；撤销快照也在移动完成后生成，而不占用对延迟敏感的按下路径。自定义连线路径按端点缓存，视口外边被裁剪，无关指针移动不再重绘整个图，鱼眼更新在拖拽期间暂停。这些修改保持最终位置和单步撤销语义，同时减少重复工作。

4.5 LIVE DEBUG 与黑板检查

Runner 把 Actor、树路径、活动节点、有序路径、节点状态、失败原因、黑板值和 Schema 信息写入进程专属临时文件，再原子发布到 `res://.godot/behavior_tree_runtime_debug.json`，避免编辑器读取半个 JSON。若快照截断，编辑器保留上一完整状态并在下一帧恢复。

编辑器按当前树过滤快照，显示路径、叶节点、路径摘要、失败菜单和实时黑板。卡片使用状态边框与标注；非活动分支可淡化到 0.24，活动路径保持 1.0。长路径和当前选择使用独立可滚动行，避免 1600×900 下重叠。游戏中没有相应覆盖 UI。

4.6 大型树显示机制

19 项功能通过统一开关系统登记，状态保存在 `Godot ConfigFile`。组合测试会启用功能、重建、全部关闭，并检查残留。

4.6.1 信息密度控制

Compact 减少卡片宽高，保留类型编码和短标题；Semantic Zoom 不改变几何，而是按缩放层级决定字段可见性；形状/图标补充颜色，可选色盲安全配色提

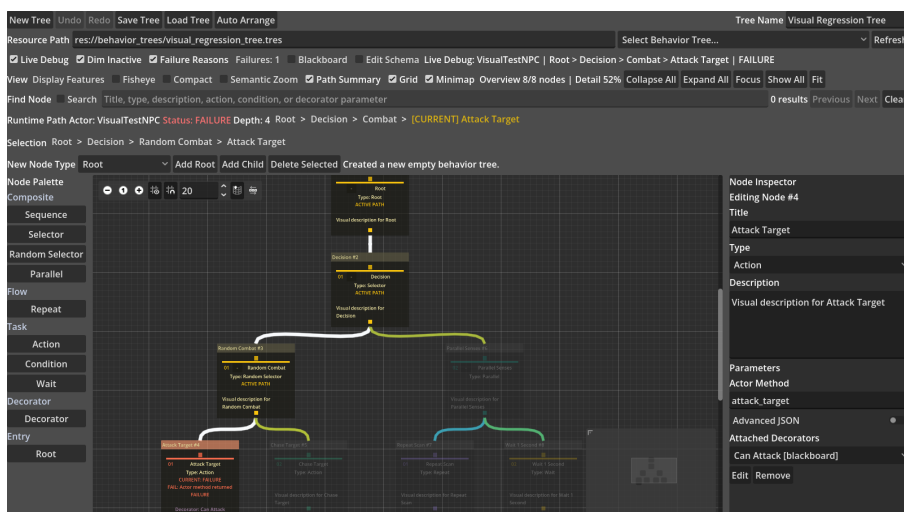
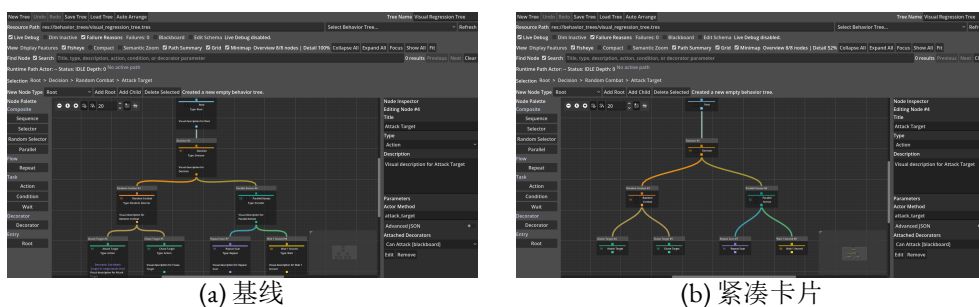


图 4.2: 仅位于编辑器中的活动路径和失败原因显示。

供冗余编码。



(a) 基线

(b) 紧凑卡片

图 4.3: 同一行为树的完整细节基线和紧凑显示。

4.6.2 焦点与结构裁剪

Fisheye 最多把鼠标所在及附近卡片放大 1.20 倍，保留远处上下文。实现不再修改 `GraphNode.scale`，而是改变内部尺寸与字体，并围绕中心补偿；关闭时恢复所有值。Collapse 隐藏后代并显示数量与下两层摘要，Focus 隔离所选分支；Stable Layout 尽量复用无冲突位置。

4.6.3 导航与概览

Search 匹配标题、类型、描述、方法和 Decorator 参数，并用 F3/Shift+F3 导航；Breadcrumb 显示祖先；Fit 计算完整树缩放，大树最低支持 0.10；230×150 小地图表示全部节点和当前视口；Multi-column 平衡宽分支，并用顺序提示保持执行优先级可见。

4.6.4 连线与运行表示

Orthogonal 使用直角边, Bundling 让同方向连线共享主干。Active Path、Branch Dimming、Decorator Badge、Path Summary 和 Failure Annotation 提供运行解释,但不修改行为树资源。它们保持可选,因为不同密度和任务对高亮与连线的偏好不同。

4.7 物理屏幕实验基础设施

屏幕尺寸 Harness 将 EDID 物理表面与原生分辨率分离。PowerShell 设备清单把每台活动 Windows 显示设备映射到 EDID 宽高、当前桌面设备和经过验证的 Godot 屏幕索引。对于宽 W_{cm} 、高 H_{cm} 的屏幕,受控画布为

$$W_c = 35W_{cm}, \quad H_c = 35H_{cm},$$

其中 W_c 与 H_c 为逻辑画布单位。图卡片保持相同逻辑尺寸,因此更大的物理表面会提供成比例增加的受控画布,而不是继承设备任意的像素密度。

真实渲染器 Harness 将实验窗口移动到指定显示器,在归一化 SubViewport 中实例化编辑器,应用同一确定性树和显示条件,并在每行记录物理元数据。每个 Session 保存不可变设备清单,以及按设备分离的原始 CSV、汇总、清单、日志和截图。组合分析会重新生成配对屏幕尺寸表与输入 SHA-256 清单;以后新增显示器时创建新 Session,不覆盖已有证据。

4.8 运行时拓扑缓存

无缓存时,Runner 会反复线性扫描数组以按 ID 找节点、查父级子节点和查所属 Decorator。可选缓存建立 ID 到节点、父 ID 到有序子节点、Owner ID 到有序 Decorator 三种索引,Condition、Action 和黑板动态结果从不缓存。

只比较资源实例和节点数不安全,因为树可以在节点数不变时原地修改。最终实现每个外部 Tick 计算一次精确签名,包括 Root ID、节点数、每个节点资源实例、节点 ID、父级、Decorator Owner 和 X/Y 排序位置。变化后重建索引,递归执行在该 Tick 内复用已验证缓存。这以一次 $O(n)$ 扫描换取正确性。

4.9 演示游戏

复杂二维竞技场提供合成树之外的行为证据。玩家可以移动、跳跃、攀爬梯子、近战、冲刺、治疗和短暂隐身;三个守卫共享同一棵 241 节点树并通过行为树组件运行。低血量时守卫撤退并治疗;响应式防御会回应玩家当前攻击窗口;方向近战检查范围与朝向;远程压制瞄准并生成可见投射物;障碍感知会选择跳过箱子而不是继续撞墙;探测到梯子附近高处玩家时则选择攀爬。较低优先

级分支还覆盖压迫推进、协同追击、最后位置搜索、返回、分层巡逻和待机变化。检测采用滞回：330 像素内获得可见目标，超过 460 像素才释放，避免边界抖动。游戏使用简单生成图形和贴图切换，把重点保留在决策逻辑并提高自动测试稳定性。

5 结果与分析

5.1 功能正确性

最终核心回归包含 574 条断言，全部通过（表5.1）。使用真实渲染器的 180 物理帧竞技场烟雾测试保持三个活动 Runner 和存活的复杂敌人。活动项目、分发模板和洁净安装项目日志均没有真实失败、脚本错误、泄漏、崩溃或非法内存访问。

表 5.1: 最终核心自动回归。

套件	通过	总数	主要覆盖
资源与运行时	153	153	结构、节点语义、Decorator、Actor、Schema、调试桥和缓存失效
编辑器 GUI	250	250	简化菜单、右键创建、图编辑、拖拽优化、历史、Schema 与 19 项开关
基础游戏	13	13	三个敌人、移动、伤害、巡逻、死亡重启与生命周期
复杂竞技场	34	34	索敌、响应式防御、近/远程攻击、跳跃、攀爬、搜索、撤退和恢复
真实 GPU 视觉	124	124	26 张当前 1600×900 截图和视觉不变量
合计	574	574	核心断言全部通过

附加套件单独报告：人体实验材料校验 91/91、固定场景真实 GPU 测试 22/22、运行时性能和拓扑安全 511/511、发布包 53/53。0.9.0 ZIP 只包含插件目录、MIT 许可证和 SHA-256 清单，并能在空 Godot 4.6 项目启用。

复杂场景证明敌人由树而非平行硬编码控制器选择行为：除追击、近战、丢失目标、搜索、撤退和治疗外，敌人还会在玩家攻击窗口选择响应式防御，在远程距离生成投射物，在感知到箱子阻挡追击时跳跃，并在探测到梯子附近高处玩家时攀爬。Actor 方法负责实际移动和伤害，但分支选择、顺序与中断来自 241 节点资源。

5.2 受控多规模显示结果

真实 GPU 实验完成全部 900 条观测和预设断言。表5.2列出五棵同生成器树的确定性几何结果。Compact 保留全部画布卡片，同时将卡片总面积降低 61.09–61.35%；Optimized Overview 同样保留全部卡片并把面积降低 90.27–90.34%。这些百分比测量表示密度，不代表人的任务成绩。

表 5.2: 五种资源规模的受控显示测量，降低比例相对 Baseline。

指标	31	61	121	241	364
画布卡片	26	51	101	202	305
Compact 面积降低	61.35%	61.30%	61.30%	61.16%	61.09%
Overview 面积降低	90.34%	90.33%	90.33%	90.29%	90.27%
Search 淡化	96.15%	98.04%	99.01%	99.50%	99.67%
Focus 卡片降低	76.92%	86.27%	87.13%	91.58%	85.90%
Collapse 卡片降低	69.23%	78.43%	89.11%	93.07%	96.39%
Baseline/Overview Fit 缩放	.214/.216	.110/.110	.100/.100	.100/.100	.100/.100
最大重叠对数	0	0	0	0	0

Search 只保留唯一目标显著并将其定位到画布内，非目标淡化比例从 31 资源的 96.15% 增至 364 资源的 99.67%。Focus 与 Collapse 在不改变资源成员的情况下缩小渲染上下文；364 节点 Focus 的非单调结果来自固定目标具有更大的局部子树，而 Collapse 随规模增大更加选择性。所有规模和条件的卡片相交为零、父子间距非负，实验前后的保存位置与从左到右执行顺序签名一致。

Fit-to-view 还暴露出一个边界。31 和 61 资源时全部 26/51 张卡片进入视口；121 资源以上 Baseline 与 Overview 均达到 GraphEdit 的 0.10x 最小缩放，主画布内只有 54 个卡片中心。因此面积降低不能解释为完整宽树一定能在一屏可读，仍需结合 Search、Focus、Collapse 和小地图控制上下文。

鱼眼结果有意保持较小：目标放大 20%，在不引起强烈位移的前提下增强局部。它不减少节点数，因此在该指标上不能与 Collapse 直接比较。

5.3 物理屏幕尺寸结果

三屏实验完成全部 270 条观测，保存全部 18 张计划截图，并且失败断言为零。表5.3报告两棵复杂受控树。覆盖率表示 Fit-to-view 后卡片中心位于画布内的比例；“图/屏幕”表示在每厘米 35 个逻辑单位下，Baseline 图边界面积与 EDID 物理表面积之比。

从 15.94 英寸到 31.55 英寸，241 节点 Baseline 覆盖率由 18.32% 增至 46.53%，364 节点由 12.13% 增至 30.82%，两者均为小屏幕值的 2.54 倍。同一物理面积变化下，两种复杂规模的图/屏幕比均降低 72.60%。这些结果量化了更大表面提供的额外上下文，但不能说明开发者能否更有效地阅读这些上下文。

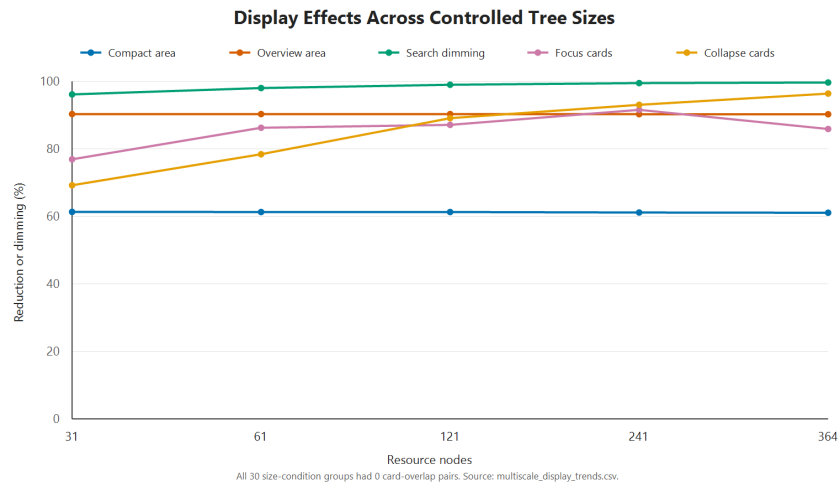


图 5.1: 五种受控规模下的密度、显著性和上下文降低。

表 5.3: 三种物理屏幕尺寸下的复杂树覆盖率。分辨率仅为审计元数据，不是实验处理。

屏幕	表面 (cm)	画布	241 覆盖率	364 覆盖率	241 图/屏幕	364 图/屏幕
BOE0CD1, 15.94 英寸	34×22	1190×770	18.32%	12.13%	116.38x	184.13x
H27T22S, 26.96 英寸	60×33	2100×1155	38.61%	25.57%	43.96x	69.56x
U32G3X, 31.55 英寸	70×39	2450×1365	46.53%	30.82%	31.89x	50.45x

显示方法自身的相对几何作用跨屏幕保持稳定：Compact 卡片面积降低范围为 61.09–61.35%，Overview 为 90.27–90.34%。Search 在全部 15/15 个“屏幕–规模”组合中都把唯一目标置于画布内；所有条件均保持零卡片重叠、非负父子间距、资源位置和执行顺序。不过，在 241 与 364 节点的三块屏幕上，Baseline 与 Overview 都达到 GraphEdit 的 0.10x 最小缩放。因此较大屏幕明显增加可见覆盖，却仍不能让完整宽树适配到单个视图。

5.4 版本化拖拽交互结果

A1–B1–B2–A2 实验获得 200 次正式拖拽；区块热身后，每个版本与树规模有 20 条观测。全部试验移动目标超过 150 像素，同步最终资源位置并保持执行顺序。表 5.4 报告完全相同的 240 采样点路径总时间，Bootstrap 区间来自各版本–规模组内固定种子 10,000 次重采样。

五种规模中四种的点估计降低，但预设总体判据还要求任何规模不得回退超过 5%。31 节点点估计回退 23.65%，所以严格判据未通过；其两个区块方向相反且区间跨零，61 与 121 节点区间也跨零。证据在 241 与 364 节点最明确，两者区间完全为正；364 节点中位数从 2,939.47 ms 降到 1,758.30 ms，降低 40.18%。这支持复杂大图固定路径处理降低，不支持所有规模都改善，也不证明真人感知流畅度。当前系统还包含拖拽优化提交之后的其他改动，因此对照测量旧版

表 5.4: 优化前与当前版本的固定路径拖拽处理时间, 中位数 [Q1, Q3]。

节点	优化前 (ms)	当前 (ms)	中位降低	Bootstrap 95% 区间
31	331.72 [317.45, 455.03]	410.17 [364.86, 428.71]	-23.65%	[-29.73%, 1.04%]
61	616.39 [468.75, 686.80]	558.17 [520.06, 594.17]	9.44%	[-17.28%, 19.09%]
121	904.23 [780.34, 1056.48]	803.08 [759.32, 850.43]	11.19%	[-0.35%, 24.81%]
241	1616.84 [1415.49, 1687.75]	1301.43 [1205.34, 1462.50]	19.51%	[9.46%, 24.73%]
364	2939.47 [2506.93, 3464.54]	1758.30 [1720.89, 1821.63]	40.18%	[29.16%, 49.21%]

与当前系统的净差异, 而不是隔离单一代码提交。

5.5 可玩 241 节点树显示结果

生成树用于隔离规模变量, 而表5.5报告 241 个资源节点实际控制可玩敌人的生态场景。图中渲染 202 张卡片, 其余 39 个 Decorator 附着在 Owner 卡片上。每个条件预热三次并记录 30 次; 循环条件顺序共生成 180 条原始观测, 并使每个条件在每个序位上恰好出现五次。

表 5.5: 可玩 241 节点行为树的客观显示结果, 降低比例均相对 Baseline。

条件	卡片数	卡片数降低	卡片面积降低	边界面积降低	淡化数
Baseline	202	0.00%	0.00%	0.00%	0
Compact Cards	202	0.00%	57.93%	5.78%	0
Optimized Overview	202	0.00%	89.48%	8.59%	0
Optimized Search	202	0.00%	89.48%	8.59%	201
Subtree Focus	32	84.16%	93.04%	63.37%	0
Context Collapse	43	78.71%	90.46%	20.99%	0

Compact Cards 保留全部卡片, 同时把卡片面积总和从 14,086,640.0 降到 5,926,318.0 px^2 。Overview 在低语义细节下同样保留全部卡片, 并把面积总和降到 1,481,579.5 px^2 。Search 淡化 202 张卡片中的 201 张, 只保留一个匹配项显著。Focus 与 Collapse 分别把当前画布限制为 32 和 43 张卡片。所有观测条件的卡片重叠均为零, 最终保存的资源位置也与实验前完全相同。这些结果说明机制能改变密度与上下文, 并且没有重现此前“所有节点叠在一起”的故障; 它们不能说明真人一定能更有效地完成任务。

为保证复现性, 实验保留了操作时间, 但因各条件执行的工作不同, 不把它们排列为共同性能名次。Baseline 完整重建的第 95 百分位最高, 为 932.64 ms; 其余条件为 327.51–534.02 ms。生态场景用于检查几何机制能否迁移, 同任务交互比较则由表5.4的受控版本化实验提供。

5.6 路径与概览结果

非活动分支淡化使用活动 alpha 1.0、非活动 0.24，即 4.17:1 对比；三种树淡化比例为 83.9%、97.5%、99.2%。首次应用从 1.902 ms 增长到 45.037 ms，稳态保持在 0.39662–0.43618 ms。说明主要成本是首次向大量卡片应用状态，而无变化更新较低。

表 5.6: 活动分支淡化与增强小地图。

节点	淡化	首次淡化 (ms)	稳态淡化 (ms)	地图覆盖	地图稳态 (ms)
31	26	1.902	0.40548	100%	0.00132
121	118	6.256	0.43618	100%	0.00140
364	361	45.037	0.39662	100%	0.00174

小地图固定为 230×150，分别表示 31、121、364 个节点。开关周期从 9.3163 ms 增长到 126.79382 ms，稳态状态更新低于 0.002 ms，区分了构建成本和持续观察成本。

5.7 视觉回归发现

视觉测试发现了数字指标无法充分表示的问题。早期鱼眼直接 Transform GraphNode，导致滚轮缩放时全部节点先缩小后放大，关闭后残留 Scale，焦点还偏到鼠标右侧。改为内部尺寸与字体、中心补偿和显式复位后，三个问题全部消失。

Search 曾在 Inspector 中选择远处卡片，却因图坐标和缩放 ScrollOffset 混用而没有居中。按照 GraphEdit.zoom 换算后修复，并加入远距目标必须进入视口的回归。按每层实际最大高度布局也消除了带多个 Decorator 的 36 节点树重叠。

其他视觉结果包括：每个关系只有一条持久线；上下方形端口清楚；运行路径与 Selection 行不重叠；复位后没有残留淡化；Decorator 摘要、Schema Key 和实时黑板类型正确。

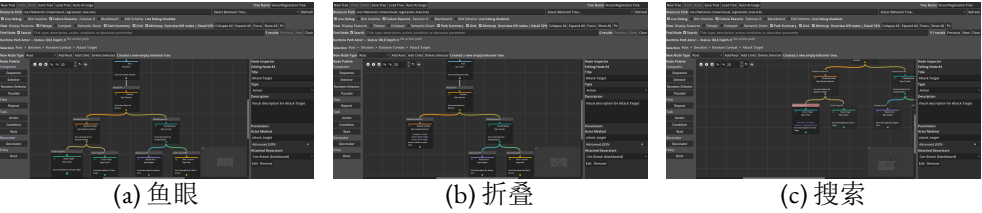


图 5.2: 焦点 + 上下文、结构裁剪和显著性导航的实现示例。

5.8 运行时缓存结果

所有缓存和非缓存实验都到达相同 success 状态。安全测试还在节点数不变时分别修改同级顺序、父级、Decorator Owner 和同 ID 资源实例，均能触发失效；整树替换和关闭缓存回退也通过。

表5.7为七次样本的 Tick 中位值。所有条件下安全缓存都降低成本，且收益随树规模显著增加。31 节点降低 15.98–26.46%，121 节点降低 52.61–56.02%，364 节点降低 76.65–78.00%。

表 5.7: 按树规模和 Runner 数统计的运行中位值。

节点	Runner	无缓存 ($\mu\text{s}/\text{Tick}$)	缓存 ($\mu\text{s}/\text{Tick}$)	降低
31	1	314.834	264.522	15.98%
31	10	376.313	276.741	26.46%
31	50	372.556	294.096	21.06%
121	1	2405.729	1140.190	52.61%
121	10	2429.358	1068.500	56.02%
121	50	2687.255	1270.075	52.74%
364	1	15963.829	3727.207	76.65%
364	10	15926.812	3635.463	77.17%
364	50	15318.540	3370.560	78.00%

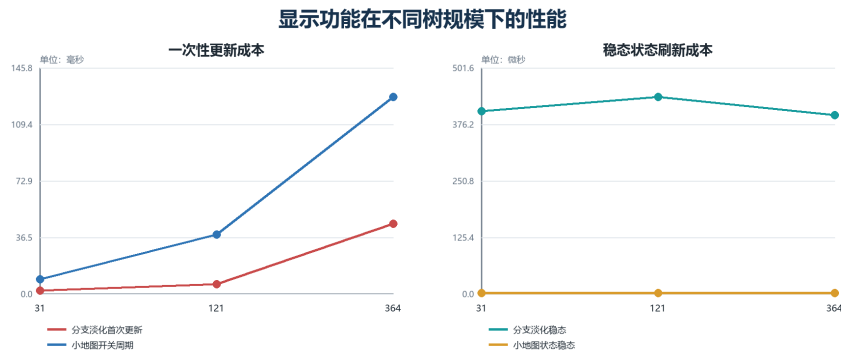


图 5.3: 开启与关闭安全拓扑缓存时的 Tick 扩展。

Runner 数增加时没有清晰单调关系，因为测试固定的是近似总 Tick 工作量，而不是让每个 Actor 在固定墙钟时间内持续运行。有效解释应是同一规模/负载内的缓存与非缓存配对，而不是跨机器生产帧率预测。

5.9 人体可用性证据边界

可选参与者工作簿包含预生成的 216 行，但实际观测为零。因此本文不报告完成时间、成功率、错误、导航、可读性、认知负荷或推断性人体结果。自动材料检查只证明未来协议与空数据保护内部一致，不属于回答 RQ2 的证据。

5.10 对应研究问题的结果

RQ1 由完整测试矩阵和复杂游戏支持：插件能够编写、序列化、校验、执行和调试非简单智能体。RQ2 在定义的仪器化层面得到回答：五种受控树规模、三种物理屏幕尺寸和可玩资源呈现可重复的密度、上下文和显著性效果，没有重叠或语义修改；固定路径处理在 241 与 364 节点最明确，但并非所有规模改善。较大物理表面使复杂树 Baseline 覆盖率增至小屏幕的 2.54 倍，但仍触及 0.10x 最小缩放。人体可用性不属于该已回答构念，仍未验证。RQ3 在当前工作负载下得到支持：精确签名缓存保持语义，并显著降低大树 Tick 中位成本。

6.1 完整系统作为研究工具

RQ1 关注系统可行性。完成的插件表明 Godot 通用图编辑和扩展 API 足以构成接近成熟工具功能基线的行为树工作流：可序列化资源、可复用 Actor 组件、有序复合节点、条件、动作、Decorator、黑板和实时执行检查。复杂竞技场是关键证据，因为只会正确画图的系统仍可能与智能体行为无关；当前检测、追击、方向攻击、搜索和恢复由资源在运行时选择，而 Actor 方法只提供物理效果。

功能广度也说明行为树编辑器不只是普通树查看器：移动同级节点会改变优先级，隐藏 Decorator 会遮蔽失败原因，切换树必须清除执行记忆，图在被检查时仍持续更新。资源、Runner 和编辑器分层降低了耦合，但集成面仍很大。因此 574 条核心断言应被视为对指定场景的证据，而不是不存在任何缺陷的证明。

不实现 Service 节点是合理范围决策。Parallel、Random、Repeat、Wait、类型化黑板和 Decorator 已能支撑复杂演示。增加另一种仿 UE 抽象会扩大运行时和编辑器组合，却不能直接回答主要研究问题。毕业项目应优先保持可辩护、经过测试的边界。

6.2 显示结果的解释

不同显示技术面向不同任务，其百分比不是统一效用分数。五种受控规模下 Overview 面积降低稳定在约 90.3%，Search 在 364 资源时淡化 99.67%，Collapse 卡片降低达到 96.39%。Collapse 适合已知可抽象子树，却会隐藏拓扑和意外运行分支；Fisheye 只放大 20%，但保留完整结构；Minimap 提供方向而不是可读细节。

物理尺寸实验增加了第二个规模维度。在每厘米逻辑密度恒定时，从 15.94 英寸增至 31.55 英寸使 241 与 364 节点的 Baseline 适配后覆盖率都变为原来的 2.54 倍。这是物理画布增加的直接几何结果，不是算法获得了意外提升。更重要的是，所有复杂树仍达到 0.10x 最小缩放。大屏幕缓解了问题，却没有消除对 Search、Focus、Collapse 和小地图的需求。Compact 与 Overview 的降低比例跨屏幕稳定，说明这些编码不依赖单一屏幕尺寸；Search 的 15/15 取景结果说明

导航操作稳健，但两项结果都不能证明感知可读性。

可玩 241 节点结果降低了几何发现只是某一种合成生成器产物的风险。其分支对应游戏开发者实际会修改的行为，例如投射物攻击、跳跃与梯子通行、搜索和巡逻，并包含 39 个会扩大真实卡片的附着 Decorator。Compact 和 Overview 仍分别把卡片面积降低 57.93% 和 89.48%，Focus 和 Collapse 则在零重叠下显示 32 和 43 张卡片。这证明显示机制可应用于有游戏意义的资源，而不是证明生态场景中的可用性已经改善。例如 32 张卡片的聚焦视图可能加速已知巡逻子系统的工作，却也可能妨碍发现更高优先级的远程攻击分支。

版本化拖拽比运行 Tick 更直接关联编写工作，因为它测量编辑器中移动卡片的操作，但规模依赖不能忽略：241 与 364 节点区间完全为正，最大树点估计降低 40.18%；31 节点点估计回退，三个较小规模的区间跨零，严格总体判据因此未通过。合理解释是延迟资源通知和快照复制属于大图优化，而不是所有规模的普遍改善；毫秒差异也不能证明真人感知流畅。

这种任务依赖性与文献一致。焦点 + 上下文平衡局部和全局 (Furnas, 1986; Lamping et al., 1995)，概览 + 细节提供稳定全局参考但会分散注意 (Cockburn et al., 2008)；多列布局的优势针对大扇出浏览 (Song et al., 2010)，不能推出所有行为树都应重排；Quilts 的路径任务结果 (Bae and Watson, 2011) 支持补充路径摘要，而不是完全替换可编辑节点图。

因此最重要的设计结果不是找到唯一“最佳方法”，而是受控组合。设计者可以用 Compact 增加密度，用 Minimap 保持概览，用 Search 定位，再用语义细节检查；运行时则用 Branch Dimming 与文字路径。独立开关也便于隔离故障，例如鱼眼残留 Transform 时可以单独关闭而不影响折叠和实时调试。

心理地图只得到部分处理。Stable Layout 和中心补偿鱼眼减少无关移动，折叠保留 Owner 与摘要；但新子树与已有位置冲突时，自动布局仍可能移动节点。相关研究强调空间连续性 (Dogrusoz et al., 2018; Misue et al., 1995)。未来应测量用户多次折叠/展开后重访节点的能力，而不只统计重叠。

6.3 LIVE DEBUG 作为可读性机制

运行检查把大树问题从一般拓扑阅读变成时序解释。Path Summary、Active Edge、Inactive Dimming、Failure Annotation 和 Live Blackboard 提供冗余证据：颜色表达位置，文字表达顺序，黑板表达决策输入。对于 Decorator 尤其重要，因为失败可以附在 Owner 上，而不是从从未执行的子节点猜测。

原子快照是编辑器和游戏分进程运行下的工程折中。它简单且可检查，但文件轮询不适合高频历史或远程目标，也只保留最新合并状态。Socket 或 Godot Debugger 协议可支持事件时间线和远程 Actor，但同步复杂度更高；对当前项目而言，稳健的最新状态比完整历史更合理。

6.4 缓存权衡

性能实验在固定工作负载下肯定回答 RQ3。索引查询替代递归执行中的重复线性扫描，因此树越大收益越高。精确签名避免了常见错误：把资源身份和节点数相同误认为拓扑未变；测试证明节点数不变的结构修改也能正确失效。

缓存并非免费。每次外部 Tick 都有 $O(n)$ 签名扫描，所以 31 节点收益只有 15.98–26.46%，明显低于 364 节点。事件驱动版本号可以达到 $O(1)$ 校验，但前提是所有修改入口都可靠递增；Godot Resource 可以直接编辑，仅靠版本号会降低安全性。未来可在编辑器变更信号和周期性精确校验之间采用混合方案。

基准还包含 GDScript 和测试框架开销，且只运行在一台 Windows 笔记本。Runner 数条件固定总工作量，因此不是 50 个生产智能体持续 Tick 的精确帧成本。结果只证明控制配对下的相对效果。

6.5 可靠性与发布

严格日志策略发现了断言可能漏掉的问题，包括异步 Timer 在场景退出后存活，以及 PowerShell 原生管道触发 Godot Console 退出期崩溃。改用节点本地状态机消除生命周期泄漏，直接启动 Godot 并指定绝对日志路径避开管道问题。调试插件本身不能制造误导错误，因此这些修复具有实际意义。

洁净安装测试验证 ZIP 边界、清单哈希和空项目启用，而不只是检查文件存在。剩余发布工作包括最终图标、Asset Library 描述和明确目标版本后的兼容矩阵。

6.6 有效性威胁

6.6.1 构念有效性

卡片面积、可见比例、物理屏幕覆盖率、字段数和淡化数测量的是表示属性，不是理解。更小的视图可能隐藏任务所需信息，更强的淡化可能降低对备选项的察觉。因此不能把受控实验 61.09–61.35% 的面积降低、2.54x 的物理覆盖率增长或可玩树 57.93% 的面积降低写成同等可读性提高；该构念需要独立人体实验。

6.6.2 内部有效性

生成树结构一致有利于比较，却可能比不规则人工树更容易搜索和布局。可玩 241 节点资源加入了具有意义的不规则内容，但仍只是一个人工工作负载。屏幕实验保持每厘米逻辑单位不变，但 EDID 尺寸存在取整，三块面板的形状和连接路径也不同，因此分辨率与计时被排除在主要屏幕尺寸推论之外。显示时间会受操作系统调度、GPU/编辑器预热和温度状态影响；三次预热、30 次重

复和循环规模/条件序位能降低但不能消除噪声。拖拽采用 A1-B1-B2-A2 平衡顺序，但 31 节点两个区块方向相反，说明时间变化仍存在；中位数、四分位数、P95 与固定种子区间用于公开而不是消除不确定性。

6.6.3 外部有效性

测试限于 Godot 4.6、Windows 和一个渲染环境。物理尺寸样本只有三台按可用性选择的屏幕，并非随机抽取的显示器范围，因此只能证明方法可追加以及三组配对观测，不能推导所有显示器的总体关系。Godot 内部 GraphEdit 层可能跨版本改变，尤其单线模式会隐藏原生子层。结果不能直接推广到移动编辑器、其他操作系统或数千节点树。具有行为意义的工作负载仍只是一个二维战斗游戏，不同开发者经验、长期生产力与其他游戏类型上的迁移仍未测量。

6.6.4 研究者与人体有效性边界

开发者同时设计系统和自动指标，可能使测试偏向已有机制。固定生成规则、预设门槛、原始行、输入哈希和报告严格拖拽判据失败有助于审计，但不能替代独立参与者。可选平衡协议未来可以测量人的任务表现；当前工作簿为零观测，所以本文结论不包含人体可用性。

6.7 实践启示

小树适合完整细节节点图；拖拽实验没有提供稳定的小树收益，31 节点估计还出现回退。中大型树的物理尺寸数据说明更多显示表面会增加上下文，却不会消除 0.10x 限制。因此应按任务提供工具，而不能假设换用大屏幕就足够：Compact/Semantic 用于概览，Collapse/Focus 用于子系统编辑，Search 用于已知目标，Minimap 用于方向，Active Path/Failure 用于诊断。稳定几何应作为安全默认，强畸变功能应可选且易于复位。

对插件开发而言，显示创新必须和编写语义一起测试：连线即使好看，不能点击断开就不完整；折叠若允许新节点无摘要地隐藏就不完整；Live Debug 若因半条消息清空正确状态就不安全。因此集成测试本身也是贡献的一部分。

7 结论

7.1 总结

本文研究如何在 Godot 4.6 中构建完整可视化行为树，同时使大型树更容易检查。项目面对两个相互连接的问题：Godot 提供通用图编辑接口，却没有完整行为树流程；传统完整细节节点图会随着节点、参数和运行标注增加而拥挤。

最终 0.9.0 插件实现可序列化树、类型化黑板、Decorator、有序与响应式复合节点、随机与并行控制、Repeat/Wait 记忆、Actor 方法 Action/Condition、可复用 NPC 组件、撤销重做和仅编辑器 Live Debug。复杂二维场景使用 241 节点资源展示索敌、响应式防御、近战与远程攻击、越障跳跃、梯子追击、最后位置搜索、撤退、治疗和巡逻。

RQ1 在定义范围内得到肯定回答：Godot 编辑器与 Resource API 可以支撑实用的编写、执行和调试流程。RQ2 在仪器化层面由五种受控树规模、三种物理屏幕尺寸和可玩资源回答：Compact 把受控卡片面积降低 61.09–61.35%，Overview 降低 90.27–90.34%，Search 淡化 96.15–99.67% 的卡片；所有条件保持零重叠、保存位置和执行顺序。200 次版本化拖拽中四种规模点估计降低，241 与 364 节点区间完全为正，364 节点中位数降低 40.18%；但 31 节点点估计回退，严格总体判据未通过，所以证据支持复杂大图而不是所有规模。270 条物理屏幕观测表明，从 15.94 英寸增至 31.55 英寸后，241 与 364 节点 Baseline 覆盖率均达到小屏幕的 2.54 倍，但所有复杂树仍触及 0.10x 最小缩放。人体理解仍未验证。RQ3 在当前工作负载下得到肯定回答：精确签名缓存保持执行结果，并将 Tick 中位成本降低 15.98–78.00%，大树收益更高。

7.2 贡献

本项目贡献了可用 Godot 插件，而不是脱离运行时的视觉原型；贡献了集成的可切换大树与运行检查机制、安全拓扑缓存，以及包含自动回归、900 条受控显示观测、200 条版本化拖拽观测、180 条可玩树观测、三台物理屏幕上的 270 条观测、固定截图、输入哈希和确定性分析的可复现证据包。它也提供重要负面边界：严格拖拽判据没有在所有规模通过，更大物理屏幕没有使任一复杂树完整适配单屏，几何压缩也不能直接报告为人的可读性已经改善。

7.3 局限

当前证据限于一台 Windows 电脑上的单一渲染环境、三台非随机物理屏幕、Godot 4.6、最多 364 节点的生成树和一个可玩 241 节点工作负载。自动结果能证明实现与表示属性，却不能证明长期编写效率或认知负荷。调试桥提供最新状态而非完整时间线。Service、可复用子树资源、Asset Library 和跨版本矩阵不在已完成范围内。

7.4 未来工作

下一项评价重点是继续追加其他物理屏幕 Session，在选定 Godot 版本、不同 GPU 和更不规则的人工树上复测受控显示与拖拽实验，并把拖拽优化提交与后续编辑器变化隔离比较。如果以后能够获得伦理许可、招募与时间，已准备的参与者协议可以额外验证 Action 定位、活动路径和 Decorator 编辑；它属于可选未来构念验证，而不是当前实验缺失的数据。

工程上可增加可复用子树接口、暂停/单步调试时间线、多 Runner Actor 选择，以及“事件版本号 + 周期精确校验”的混合缓存。显示研究可以加入增量动画与可配置兴趣度函数，并测量重访和心理地图。最后还应验证选定 Godot 版本并准备 Asset Library 上架。

7.5 最终结论

本项目说明毕业设计可以超越对既有编辑器的简单模仿。通过结合可执行语义、仅编辑器解释、可切换焦点与复杂度控制、严格回归和明确证据边界，该插件既是实用 AI 编写工具，也是后续研究可读性可视化编程的平台。

参考文献

- Colledanchise, M. and Ögren, P. (2018). *Behavior Trees in Robotics and AI: An Introduction*. CRC Press. doi:10.1201/9780429489105.
- Furnas, G. W. (1986). Generalized fisheye views. *Proceedings of CHI '86*, 16–23. doi:10.1145/22339.22342.
- Lamping, J., Rao, R. and Pirolli, P. (1995). A focus+context technique based on hyperbolic geometry for visualizing large hierarchies. *Proceedings of CHI '95*, 401–408. doi:10.1145/223904.223956.
- Cockburn, A., Karlson, A. and Bederson, B. B. (2008). A review of overview+detail, zooming, and focus+context interfaces. *ACM Computing Surveys*, 41(1), 1–31. doi:10.1145/1456650.1456652.
- Song, H., Kim, B., Lee, B. and Seo, J. (2010). A comparative evaluation on tree visualization methods for hierarchical structures with large fan-outs. *Proceedings of CHI 2010*, 223–232. doi:10.1145/1753326.1753359.
- Bae, J. and Watson, B. (2011). Developing and evaluating Quilts for the depiction of large layered graphs. *IEEE TVCG*, 17(12), 2268–2277. doi:10.1109/TVCG.2011.187.
- Dogrusoz, U. et al. (2018). Efficient methods and readily customizable libraries for managing complexity of large networks. *PLOS ONE*, 13(5), e0197238. doi:10.1371/journal.pone.0197238.
- Misue, K. et al. (1995). Layout adjustment and the mental map. *Journal of Visual Languages and Computing*, 6(2), 183–210. doi:10.1006/jvlc.1995.1010.
- Reingold, E. M. and Tilford, J. S. (1981). Tidier drawings of trees. *IEEE Transactions on Software Engineering*, SE-7(2), 223–228.
- Plaisant, C. et al. (2002). SpaceTree. *Proceedings of IEEE InfoVis 2002*, 57–64.
- Shneiderman, B. (1996). The eyes have it. *Proceedings of IEEE Visual Languages 1996*, 336–343.

Godot Engine contributors (2026a). GraphEdit class reference. https://docs.godotengine.org/en/stable/classes/class_graphedit.html.

Godot Engine contributors (2026b). EditorPlugin class reference. https://docs.godotengine.org/en/stable/classes/class_editorplugin.html.

Epic Games (2026). Behavior Tree in Unreal Engine: Overview. <https://dev.epicgames.com/documentation/en-us/unreal-engine/behavior-tree-in-unreal-engine---overview>.

功能参考

表 A.1: 最终编辑器的独立显示功能。

功能	默认	作用
Fisheye / Focus+Context	开	鼠标邻域最多放大 1.20x。
Subtree Collapse / Expand	开	隐藏后代并显示数量与两层摘要。
Compact Mode	关	仅保留类型编码和短标题。
Active Path Highlight	开	强调当前运行路径。
Non-active Branch Dimming	关	Live Debug 时降低非活动分支 Alpha。
Multi-column Layout	关	平衡宽同级分支并保持顺序提示。
Enhanced Minimap	开	230×150 概览和视口标记。
Semantic Zoom	关	不改变图几何地切换信息细节。
Path Summary View	开	显示 Actor、状态、深度和有序路径。
Decorator Condition Badges	开	在 Owner 卡片总结条件。
Search + Highlight	开	搜索节点/Decorator 并淡化不匹配项。
Orthogonal Edges	关	使用直角连线。
Edge Bundling	关	密集区域共享主干。
Stable Incremental Layout	关	布局时复用无冲突位置。
Breadcrumb Navigation	开	显示并导航祖先。
Failure Reason Annotation	开	显示节点失败标签与摘要。
Shape / Icon Type Encoding	开	使用形状和图标补充颜色。
Accessibility Palette	关	色盲安全配色与键盘导航。
Single Connection Rendering	开	显示一条可命中的自定义持久线。

复现与证据索引

以下路径均相对于仓库根目录：

活动项目： `testgame/testgame/`

分发插件： `visual_scripting/addons/behavior_tree_editor/`

复杂树： `testgame/testgame/behavior_trees/complex_guard_validation_tree.tres`

测试脚本： `testgame/testgame/tests/`

多规模显示数据： `testgame/testgame/test_results/multiscale_display_raw.csv`

多规模拖拽数据： `testgame/testgame/test_results/multiscale_drag_raw.csv`

多规模汇总： `testgame/testgame/test_results/multiscale_display_summary.csv` 与 `multiscale_drag_summary.csv`

分析与报告： `research/display_optimization/analyze_multiscale_experiment.py` 与 `multiscale_experiment_results.md`

性能数据： `testgame/testgame/test_results/runtime_profile.csv`

视觉截图： `testgame/testgame/test_results/visual/`

人体实验协议： `research/display_optimization/human_comparison_study_protocol.md`

人体实验工作簿： `research/display_optimization/behavior_tree_human_comparison_study.xlsx`

人体实验分析： `research/display_optimization/analyze_human_study_results.py`

可玩树原始数据： `testgame/testgame/test_results/complex_display_experiment_raw.csv`

物理屏幕证据： 实验根目录为 `research/display_optimization/physical_screen_size_experiment/`；其中 `data/2026-08-23_three_displays/Session` 包含 `combined_raw.csv`、`screen_size_comparison.csv` 与 `report_zh.md`。

发布包： `dist/behavior-tree-editor-0.9.0.zip`

真实视觉回归必须使用 `-rendering-method gl_compatibility`，不能使用 `Dummy Headless`，因为 `Dummy` 无法提供所需 `SubViewport` 纹理。

可选未来参与者实验

协议在可玩 241 节点树上包含六种显示条件和每种三个任务。参与者将先完成三项标准化练习，再进行 18 个平衡正式试验。六组 Action/路径目标和六个 Decorator 目标与条件顺序共同平衡，使 12 名参与者中每个方法-目标组合恰好出现两次。工作簿包含 216 个空行，记录上限完成时间、误选、Zoom、Pan、成功率、可读性和认知负荷。它仅用于可选的未来构念验证，不是当前论文结果来源；使用前必须获得所需伦理许可并与导师确认流程。

初稿完成清单

正式提交前，作者应：替换身份与导师占位符；按学校政策修改声明并如实说明 AI 协助；在 Tabula 核实截止时间和词数规则；与导师确认可选参与者协议是否保留为未来材料；请导师审阅文献、研究问题和证据边界；按院系要求执行 Turnitin；用模板编译并逐页检查；加入最终词数和要求的仓库/归档信息。